Hexapod Multi-Terrain RL: Lab Meeting Report

Valero Lab

February 2026

Project: Hexapod locomotion with PPO and optional sCPG
Focus: Training on varied terrain (flat, rough, steps) and evaluation pipeline

---

1 Goal

- Train a single policy that can walk on **flat**, **rough**, and **stepped** terrain.
- Use the same reward structure and observation space as the existing flat-terrain MLP runs (e.g., mlp_run_005) so that improvements are comparable.
- Provide a clear evaluation and visualization pipeline: metrics plus videos with terrain visible and a zoomed-in camera for presentations.

2 Environment and Terrain

2.1 Robot and Base Setup

- **Model:** MuJoCo hexapod with 6 legs, PD-controlled joints (abd and flex per leg).
- **Task:** Circular path following (path_radius 0.5 m) with path-heading and path-proximity rewards; velocity and smoothness terms as in `config_v7_mlp_clean`-style configs.
- **Episode length:** 1000 steps (frame_skip 5).

2.2 Terrain Implementation

- **Asset:** `hexapod_with_terrain.xml` — same robot as `hexapod.xml`, but the floor is a **32×32 heightfield** (MuJoCo `hfield`) so we can change elevation at reset.
- **Terrain types:**
  - **Flat:** Constant zero elevation.
  - **Rough:** Random smooth noise; configurable scale and height range (`terrain_rough_scale`, `terrain_rough_min_height`, `terrain_rough_max_height`). Training uses ±2 cm; evaluation videos use ±6 cm so the terrain is clearly visible.
  - **Steps:** Stair-like profile along one axis; `terrain_step_height`, `terrain_step_length`, `terrain_num_steps` (e.g., 2 cm height, 8 cm length, 5 steps).

- **Sampling:** Each episode reset draws a terrain type. With `terrain_type: random`, the type is chosen uniformly from {flat, rough, steps}. An optional **terrain curriculum** (flat only, then flat+rough, then all three) can be enabled via config and a training callback.

2.3 Observation and Rewards

- Observations include joint positions, joint velocities, contact, phase, previous action, path heading error, and torso velocity (no torso quat in terrain config).
- Rewards: path heading (3.0), path proximity (2.0), target velocity (28.0), smoothness (2.5), jerk (0.35), lateral/yaw/tilt penalties, stance and idle penalties, etc., matching the flat-terrain MLP setup so the policy is incentivized for stable, directed walking on all terrains.

3 Training Setup

- **Config:** `configs/config_terrain.yaml`
- **Algorithm:** PPO (Stable-Baselines3), MLP policy [256, 256] for both policy and value.
- **Hyperparameters:** 2048 steps per env, batch 64, 10 epochs, LR 3×10^{-4}, gamma 0.99, GAE lambda 0.95, clip 0.2.
- **Training length:** 300k timesteps per run.
- **Envs:** 2 parallel envs (DummyVecEnv).
- **Checkpoints:** Every 50k steps; final model saved as `final_model.zip`.
- **Eval during training:** Every 10k steps, 5 episodes (on whatever terrain the env samples).

4 Runs

Run	Config	Target steps	Status	Notes
terrain_run_001	config_terrain.yaml	300,000	Complete	303,104 steps, 302 episodes
terrain_run_002	config_terrain.yaml	300,000	Complete	Same config; second replicate

4.1 terrain_run_001

- **Path:** `runs/terrain_run_001/`
- **Training summary:** total_timesteps 303,104, total_episodes 302, mean_episode_length 1001.
- **Checkpoints:** progress at 28k, 57k, 86k, 114k, 143k, 172k, 200k, 229k, 258k, 286k, 303k steps; `final_model.zip` available.
- **Eval outputs:**
  - `eval/` — default eval (metrics + videos if enabled).

– `eval_visible_terrain/` — same but with terrain heights overridden for visibility (e.g., rough ±6 cm, steps 5 cm height) and renderer recreated so the heightfield is shown; videos include a "Terrain: flat/rough/steps" overlay.

4.2 terrain_run_002

- **Path:** `runs/terrain_run_002/`
- **Training summary:** total_timesteps 303,104, total_episodes 302, mean_episode_length 1001 (same as run_001).
- **Eval:** Standard eval in `eval/`; can generate `eval_visible_terrain` the same way as run_001.

5 Evaluation

5.1 How evaluation is run

From project root (`hexapod_training_new`):

```
python evaluate.py <path_to_model.zip> --config <config.yaml> --output-dir
    <dir> --num-episodes N --save-video
```

Example for terrain_run_001 with visible terrain and videos:

```
python evaluate.py runs/terrain_run_001/checkpoints/final_model.zip \
  --config runs/terrain_run_001/config.yaml \
  --output-dir runs/terrain_run_001/eval_visible_terrain \
  --num-episodes 6 --save-video
```

5.2 Terrain cycling for videos

When the env uses the terrain heightfield, the evaluator cycles terrain per episode: episode 0 → flat, 1 → rough, 2 → steps, 3 → flat, etc. Each episode's video is labeled (e.g., "Terrain: rough") and the ground is overridden so rough/step geometry is visible in the render.

5.3 Metrics recorded

- **Episode-level:** total reward, episode length.
- **Step-level (aggregated):** mean/max forward velocity, mean torso height, mean energy proxy (sum of squared actions), termination reasons.
- **Output:** `evaluation_summary.yaml` in the output dir (mean_reward, std_reward, mean_length, mean_forward_velocity, max_forward_velocity, mean_torso_height, mean_energy, num_episodes, termination_counts). Exact values depend on run and number of episodes; run the command above to regenerate.

5.4 Visualization

- **Videos:** Step, (x, y), forward distance, forward velocity, and terrain label overlaid on each frame. Saved under `<output-dir>/videos/` (e.g., `episode_1.mp4`).

- **Trajectory plots:** For each episode, `episode_N_trajectory.png` with (x, y) trajectory and forward distance/velocity vs step.
- **Camera:** The terrain asset defines an `eval_cam` camera (targetbody on torso, close offset, fovy 42°) used automatically during `render()` so evaluation videos are zoomed in for lab meetings.

5.5 Results snapshot (terrain runs)

- **Training:** Both terrain_run_001 and terrain_run_002 reached ∼303k steps with mean episode length 1001 (episodes run to max length; no early termination).
- **Evaluation:** Summary statistics are written to `eval/evaluation_summary.yaml` and `eval_visible_terrai`
  To get a fresh snapshot for the meeting, run the evaluation command above, then read the printed "EVALUATION SUMMARY" block or the saved `evaluation_summary.yaml`.

6 Implementation Notes (for reproducibility)

- **Terrain in reset:** In `HexapodEnv.reset()`, when terrain is enabled, we compute heights with `_generate_terrain_heights(terrain_kind)`, write to `model.hfield_data`, and set `_terrain_changed = True`.
- **Rendering:** When `_terrain_changed` is True, the next `render()` call closes and recreates the MuJoCo renderer so the updated heightfield is drawn. The same render path uses `eval_cam` when present (terrain XML).
- **Config:** All terrain-related options (terrain_type, rough/step parameters, terrain_curriculum, terrain_curriculum_stages) are in `configs/config_terrain.yaml` and passed through `make_hexapod_env()`

7 Summary for the lab meeting

- **What we did:** Added a heightfield-based terrain system (flat / rough / steps) and trained two full runs (terrain_run_001, terrain_run_002) with 300k steps each. The policy is evaluated across all three terrains with metrics and zoomed-in, terrain-labeled videos.
- **Where to look:** `runs/terrain_run_001/` and `runs/terrain_run_002/` for training summaries and checkpoints; `eval_visible_terrain/videos/` for presentation-ready clips; `evaluation_summary.` in each eval dir for numeric results.
- **Next steps (optional):** Enable terrain curriculum, tune rough/step difficulty, or compare terrain policy vs flat-only policy on a fixed test set of terrains.

8 File reference

Item	Path
Terrain config	`configs/config_terrain.yaml`
Terrain XML	`assets/hexapod_with_terrain.xml`
Env + terrain logic	`envs/hexapod_env.py`
Training script	`train.py`
Evaluation script	`evaluate.py`
Run 001	`runs/terrain_run_001/`
Run 002	`runs/terrain_run_002/`
This report	`reports/terrain_training_lab_meeting_report.md`